Multimedia Tools and Applications (2020) 79:13841–13868
https://doi.org/10.1007/s11042-019-08592-z

FPGA based generic RO TRNG architecture for image confusion

Sivaraman R[1] · Sundararaman Rajagopalan[1] · Rengarajan Amirtharajan[1]

Received: 26 March 2019 / Revised: 3 November 2019 / Accepted: 13 December 2019 /
Published online: 3 February 2020
© Springer Science+Business Media, LLC, part of Springer Nature 2020

Abstract
True Random Number Generator (TRNG) has become a central element of today's secure communication. TRNGs developed through FPGA implementation have a significant role in a number of applications in multimedia communication. Ring Oscillator (RO) has been adopted extensively for developing device-independent TRNG structures. The proposed design is a generic TRNG architecture based on an arrangement of ROs with post-processing. This TRNG was initially tested on Altera Cyclone II FPGA which consumed only 64 inverters to produce good randomness. Also, the device-independent capability of this design has been verified by implementing it on other two FPGA families Xilinx Artix – 7 and Microsemi Smart Fusion2. True randomness was also verified by conducting restart experiment, and the statistical properties of TRNG were evaluated through entropy analysis and NIST SP 800–22 tests. The proposed TRNG yields 26.640650 Mbps as throughput with a sampling clock of 27 MHz. Going beyond the regular key generation utility, the evolving random bits of TRNG were packed suitably to perform confusion of grayscale images producing a near-zero correlation of pixels, thereby extending the application of TRNG to image encryption.

Keywords TRNG · FPGA · Ring oscillator · Image encryption

1 Introduction

Conventionally, the real-time high-speed cryptographic algorithms were implemented on Application Specific Integrated Circuits (ASICs) [39]. During recent years, a lot of security architectures are realised on Field Programmable Gate Arrays (FPGAs) platform [37]. Beyond

✉ Sundararaman Rajagopalan
raman@ece.sastra.edu

[1] School of Electrical & Electronics Engineering, SASTRA Deemed University, Thanjavur 613 401, India

the common advantages of FPGA such as reprogramability, reusability, hard-wired capability, faster time to market, etc., significant benefits with respect to cryptographic primitives like algorithm agility, algorithm modification, high throughput, easy prototyping, inbuilt DSP blocks and memory controller, analog, digital conversions, and System on Chip (SoC) availability make the FPGA as a fruitful platform for the developers to create cryptographic architectures and cryptosystems [56].

The security of any cryptography centric algorithm relies upon the key. Key is a fundamental object which adds strength to the complexity of security procedure followed by any security standard or algorithm [13]. To achieve attack-resistant security, keys must be random and unpredictable in nature which is a well-known fact. To accomplish this attribute, Random Number Generators (RNGs) play a pivotal role in which two types of RNGs are employed in practice namely Pseudo Random Number Generator (PRNG) [6] and TRNG [11]. Though PRNG produces random numbers with acceptable entropy, it incorporates mathematical expression or fixed reproducible architecture mostly based on software to generate random sequences which are vulnerable to side-channel attacks [40]. Hence, the central attention towards TRNG is obvious owing to its unpredictable nature.

TRNG employs physical mean such as noise at circuits, radioactive decays, cosmic rays, environment noise and jitters. In other words, wherever random phenomena are observed, TRNG can be developed with a proper mechanism of acquisition. Invariably, the generation of true random numbers is governed by the three elements namely entropy source, harvesting mechanism and post-processing [49]. Entropy source is a random source from where the required randomness for the generation of the true random numbers is captured. A suitable harvesting mechanism can accomplish the extraction of randomness from the entropy source. Post-processing is a kind of randomness enhancement applied to the raw true random bits to improve their statistical characteristics by reducing their correlation among the bits.

In this proposed work, Ring Oscillator (RO) based TRNG has been suggested with a new architecture with enhanced entropy for its utilisation in image security. On most of the occasions, the utility of TRNG is restricted to Session key generation, Nonce generation, S-Box generation, etc. In this work, TRNG has been utilised for scrambling the pixels (confusion) of the images. The motivation for this experiment is the utilisation of TRNG for diffusion alone in earlier works [43] and the requirement of new encryption techniques apart from chaotic maps or attractors [22, 60]. Besides the statistical analysis for evaluating the strength of TRNG, confusion capability of TRNG bits have been presented with the correlation analyses of the scrambled images. The manuscript addresses the design strategies of the proposed RO TRNG along with the image confusion application.

2 Related works

Various scrambling schemes are being utilised in image encryption algorithms for executing the confusion process. In general, confusion operation of any image can be achieved by utilizing the indices of sorted random sequences from any Random Number Generator (RNG). Many approaches have been proposed in image encryption to accomplish the confusion operation. In a method suggested by Artiles et al., traditional block cipher algorithm Rijndael with a counter mode of operation was adopted for pixel confusion. This enhanced scheme required fewer rounds of operation to yield adequate security which was further evaluated through standard metrics [2]. Hasheminejad and Rostami proposed a bit swapping technique to

perform image confusion in which Piece Wise Linear Chaotic Map (PWLCM) has been utilised [22]. Naseer et al. demonstrated row, and column permutation assisted confusion processes with three-dimensional chaotic equations to lower the correlation among adjacent pixels [36]. Ramalingam et al. extended the row and column permutations with a different perspective in which they proposed a three-phase confusion process which contains interplanar row permutation, interplanar column permutation and interplanar pixel shuffling using combined chaotic map generated random sequences [44]. The random sequence evolving from Modified Chua's Circuit (MCC) has been utilised by Arpaci et al. for performing confusion [1]. Xingyuan et al. recommended a Zigzag permutation on an image with Lü chaotic system to yield near-zero correlation [60].

TRNGs can be an excellent source to perform scrambling in image security. True random numbers generated by any TRNG must satisfy the fundamental properties, namely high entropy, equidistribution of random bits and unpredictability. To achieve these characteristics, several types of TRNG designs have been proposed so far using efficient entropy sources namely white noise [3], Johnson noise [27], digital logic gates [12], Linear Feedback Shift Register (LFSR) and Cellular Automata Shift Register (CASR) [52], digital circuit artifacts [14], Phase Locked Loop (PLL) [17], open-loop structures [9], BRAM write collisions [21], chaotic oscillator [15], and Digital Clock Managers (DCM) [26]. In spite of these sources, RO is the unique and promising entropy source among all the solutions owing to its characteristics namely low power consumption, a lesser number of hardware elements and device independent capability. However, device independency forms a TRNG suitable to be a part of any security architecture on FPGAs. NIST and Diehard tests have been the main statistical tests for evaluating these TRNG models.

Tsoi et al. presented a device-independent design of RNG which produced pseudo-random as well as true random numbers wherein the TRNG bits were used as the seed of PRNG. The ideology behind this work was sampling the high-frequency clock generated through an oscillator by a slow frequency clock generated by an external oscillator. This design was implemented on Xilinx Virtex XCV300E – 8 FPGA which occupied 15 slices at a bit rate of 225 bps. This TRNG is prone to tampering attacks because of the presence of external oscillator [54]. Kohlbrenner and Gaj also suggested a device-independent TRNG using Complex Logic Blocks (CLB) based on the combination of self-activated ROs, sampler and control units [29]. The principle of this TRNG is to make use of the on-chip intrinsic jitters produced by CLB ROs and subsequent sampling. This approach required a total propagation delay of 3.7 ns to complete one cycle of oscillation, which meant a frequency of 130 MHz. XOR tree was the post-processing to alleviate the bias problem and further the throughput was reduced to 0.5 Mbps. This TRNG was implemented on Xilinx Virtex XCV1000 FPGA which had shortcomings in the placement of ROs in CLBs and the selection of RO frequencies.

Mathematical modelling of RO based TRNG was proposed by Sunar et al. where phase jitters generated by RO were considered as a source of randomness [51]. This TRNG has four modules, namely Oscillator rings, XOR tree, sampler and resilient function as post-processing. This work also addressed a "coupon collector" problem with identical inverter rings used for constructing TRNG. The random bits were generated at a throughput of 2.5 Mbps yielding 0.97 as entropy. RO TRNGs can be designed with various numbers of inverters and rings. In this context, Schellekens et al. continued the work of Sunar et al. on the FPGA platform [48]. Three versions of RO based TRNG were illustrated in this work. Initially, TRNG proposed in [51] was implemented on Xilinx Virtex XC2VP30 FPGA which consumed 1664 slices. The modified version of this TRNG which occupied 565 slices on FPGA was designed using 110

rings of ROs with each ring cascading 3 inverters. Finally, RO design comprising 210 rings was suggested to yield good entropy results utilising 973 slices. This work yielded a throughput of more than 2 Mbps at a sampling frequency of 40 MHz.

Wold and Chik enhanced the RO based TRNG presented in [51] further using Altera Cyclone II FPGA by adopting a sampling frequency of 100 MHz to achieve the throughput of 100 Mbps [59]. This enhancement was carried out on sampling by putting an additional D flip flop at each edge of RO rings to avoid the high transition effects of RO and metastability caused in D flip-flop. In this architecture, two versions of TRNG were designed, one with 25 rings of ROs that consumed 83 logic elements and the other being 50 rings of ROs which utilised 167 logic elements. Restart experiment had also been conducted to prove the true randomness of enhanced TRNG.

Varchola and Drutarovsky recommended a high entropy source-based bi–stable Transition Effect Ring Oscillator (TERO) for TRNG. This design had five inverters, two XOR gates and AND gates. It had both RO and TERO modes producing jittery oscillations which were compared based on the intrinsic noise and sensitivity of oscillator. This TRNG consumed 9 logic functions and 8 LUTs of Xilinx Spartan 3E FPGA in which the throughput was not reported [57]. Jessa and Matuszewkski suggested an RO based TRNG wherein the combined independent jitter generators were the sources of entropy. De – synchronisation technique (XOR tree) was applied to extract the randomness from independent random sources. The enhancement in entropy was carried out by using Auxiliary Source of Randomness (ASR) with ROs. This proposed design was implemented on Xilinx Virtex – 5 FPGA while the random bits were acquired for various sampling frequencies such as 100, 150, 200 and 250 MHz. The maximum throughput achieved in this work was 20 Mbps [24]. To continue this work to yield better statistical properties, an extra delay line was added in RO based combined TRNG [25]. Carry4 logic (carry chain primitive) in Virtex – 5 FPGA was used to enhance the design. However, Carry4 module is an inbuilt IP component in Virtex – 5 FPGA which indicates that this approach is a device-dependent one. Noticeably, it strengthened the TRNG design to be resistant to frequency injection attacks. NIST and restart experiments were conducted on this TRNG which was constructed using 50 rings achieving a maximum throughput of 12.5 Mbps through a sampling frequency of 100 MHz.

Marghescu et al. modified the RO based TRNG work reported in [51] with some changes on inverters with lengths of 3, 5 and 7 instead of a fixed 3. This design was implemented on Xilinx Zynq System on Chip (SoC) FPGA wherein the true random numbers were generated using Zynq 7000 FPGA with an acquisition in ARM Cortex A9 for further enhancement through Von-Neumann post-processing. NIST statistical test suite was implemented on the Zynq SoC platform to verify the correctness of TRNG [33]. Łoza et al. presented a RO based TRNG with SHA – 256 as a post-processing mechanism. This method has overcome the problem associated with biasing and correlation among the bits. Frequency drifts produced by inverters were sampled using D flip-flop with crystal oscillator frequency and XOR tree utilised for extracting the randomness from the source. Eight stages RO was designed and implemented on Xilinx Viertex – 5 FPGA to carry out the experiment and analysis. The design consumed 2540 of slice registers and 2467 of slice LUTs. The maximum operating frequency in this design was 263 MHz to yield a throughput of 36 Mbps [31].

In subsequent work, Marghescu et al. proposed yet another SoC solution for RO TRNG with 3, 5, 7 and 11 inverters in four rings using Xilinx Zynq SoC [34]. The design of TRNG

was divided into two sections wherein the first section acquired the RO results and performed VNC post-processing. In another section, the raw RO results were taken directly and XORed with the results obtained in the first section. The same work was presented in [32] in which the NIST test suite was implemented on the target device itself to evaluate the true random bits. A study of frequency characteristics and implementation strategy of RO based TRNG on Xilinx Spartan – 6 FPGA was illustrated by Barbareschi et al. [4]. Their analysis stated that the frequency of oscillation does not only depend on design parameters such as a number of inverters, rings and placement of inverters but also affected by other working conditions including temperature and ageing. This TRNG was designed with variable inverter stages chosen between 4 and 8 for the realization. The outcome of this study revealed that the working temperature is directly proportional to signal propagation delay. High temperature causes performance degradation in terms of reduction in speed and ageing, which also reduces the lifetime of the device. This study also helps to analyse the RO based TRNG when the TRNG is device-dependent [4]. In addition to FPGA centric TRNGs, several ASIC based TRNGs have also been presented in previous works [7, 8, 20, 38, 41, 47].

On analysing the earlier works, following are the observations:

- Harvesting mechanisms in RO based TRNGs is through De – synchronisation or jitter counting methods
- Device independency was not analysed by performance analysis of the same design in different devices
- Power dissipation was not reported in most of the works
- Application was restricted to secure key generation in cryptography
- Storage of TRNG bits to memory after extraction has not been extensively discussed as it becomes a significant unit
- Maximum entropy achieved was 0.97 for RO based TRNGs with post-processing
- Maximum throughput achieved was 100 Mbps in RO TRNGs using PLLs

The proposed method of TRNG for image confusion also uses RO as an entropy source in which the design comprises 8 rings of ROs with 64 inverters. This work proposes a new harvesting mechanism called chaotic selection of jittery oscillations through the random sequence selector. This improves the selection of RO frequencies thus producing good entropy results. To enhance the design further, VNC and LFSR are used as post-processing through which a noticeable improvement in the statistical properties of TRNG was observed. The device-independent capability of the proposed TRNG has been proved by implementing the design on three different FPGA architectures namely Altera Cyclone II, Xilinx Artix – 7 and Microsemi SmartFusion2. The significant features of the proposed model are:

- No PLLs were used
- Independent TRNG irrespective of any FPGA vendor and family
- No complex design constrains were used
- Chaotic selection of random bits to avoid synchronization issues
- Simple post-processing methods
- Optimised entropy source with 64 inverters
- Tamper-resistant TRNG architecture without external influences for driving ROs
- Utility of random numbers for scrambling the pixels of an image

3 Proposed RO TRNG architecture

Strength of any TRNG depends upon the nature of entropy source which is supposed to exhibit very high randomness. The main principle of the generation of true random numbers is adopted from Sunar et al.'s work [51]. However, enhancement in the proposed design has been made on the entropy source and harvesting mechanism of TRNG with a target of an increase in throughput. From the earlier findings, limiting the number of inverters in a ring produces oscillations in different frequencies and vice versa [38]. When the oscillating frequency increases, more unstable jittery oscillations were obtained wherein the jitters are promising agents for extraction of randomness. These chaotic oscillations exhibit true randomness which cannot be predicted as well as replicated. To extract the jitters, traditional mechanisms namely XOR tree and jitter clock counters have been used in the earlier works.

Proposed modified TRNG (Fig. 1) comprises 8 RO rings wherein each ring is made of an odd number of inverters beginning from 1 to 15. This entropy source generates 8 different jittery clocks at sampling moment through the switching of inverters and subsequently, the generated clocks are inputs of the Random Sequence Selector (RSS) which uses Arbitrary Signal Generator (ASG) to produce aperiodic signals in order to select the jittery clocks. D Flip Flop (DFF) at a sampling frequency H_{clk} captures the jitters before it reaches the ASG. At the post-processing stage, these true random bits are stored in 128-bit registers to apply either VNC or LFSR corrections. Another different clock called S_{clk} is used for sampling, post-processing and storing true random bits in on-chip memory. The main principle of this scheme is to extract the true randomness from the ROs with a mixture of prime length and non-prime inverters. Selection of RO output using a sampling clock is the main part of the proposed architecture.

3.1 Entropy source (RO)

Ring Oscillators are constructed with odd number of inverters. The clock time period generated by any RO is given by

$$T = 2 \sum_{i=1}^{k} d_i \tag{1}$$

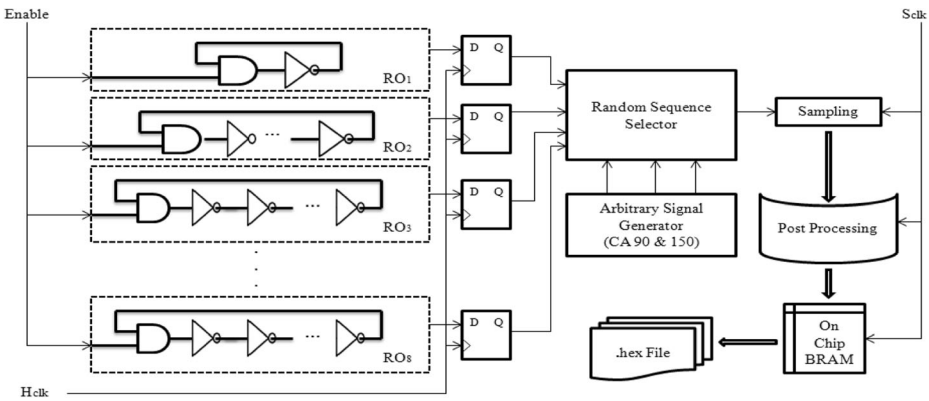

Fig. 1 Block diagram of proposed TRNG

Where d_i is the delay of each inverter in the RO [19]. The local and global jitters in RO are the main sources of entropy in TRNGs. As per the local jitter model, the delay d_i is modulated based on local deterministic jitter (Δd_{Li}) and local Gaussian jitter (Δd_{Gi}). With Di as the inverter static delay, the d_i in Eq. (1) is given by,

$$d_i = D_i + \Delta d_{Li} + \Delta d_{Gi} \tag{2}$$

The global jitter is a function of power supply fluctuation and temperature variations which in turn causes Gaussian global jitter (Δd_{GG}) and deterministic global jitter (Δd_{GD}). The delay variation in inverters can be brought in ASIC design easily in which the CMOS logic design of inverters can provide options to vary the rise time and/or fall time delays. This delay variation creates an impact in the jitters which is the prime reason for generating more random output bits from the inverters of RO. Designing RO in FPGA has its own limitations such as fixed inverter primitive without options to alter the delays, simplification of inverter circuit using LUTs in logic elements, difficulties in manual routing in complex RO circuits, etc. However, RO can be designed comfortably with FPGAs and also they can generate true random bits through the LUT realisation of coupled inverters of RO. Figure 2 shows a collection of 8 ROs with each RO made of 3 identical inverters. The timing simulation of Cyclone II FPGA based RO with three inverters is shown in Fig. 3. This simulation demonstrates an accumulation time of approximately 77.03 ns after which the two-level oscillations begin. Periodic oscillations were observed in this RO with a clock frequency of 20.1822 MHz. However, the entropy values estimated from 20 sets of 1,31,072 random bits extracted by a random selection of the outputs of ROs lie between a mediocre 0.82 to 0.9 (Fig. 4) which are certainly not convincing figures.

On observation, an identical number of inverters used in a RO was unable to fetch consistent and high entropy [38]. In addition, nearly 100 or 200 ROs are required to get statistically efficient random numbers when an identical number of inverters in each RO is preferred [51]. Hence, the proposed TRNG uses a collection of RO structures wherein each RO was developed with a variable number of inverters (Fig. 5). The motivation of this modification is to investigate the possibility of good entropy yielding TRNG with limited resources. The ROs in the structure have employed an odd number of inverters in increasing order. Due to the variable number of inverters in each ring, the frequency variations have been inferred as per timing analysis shown in Fig. 6. Frequency of RO can be calculated using the expression (3),

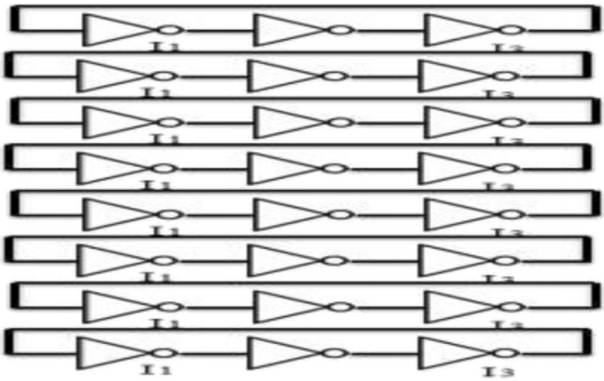

Fig. 2 RO collection with three inverters

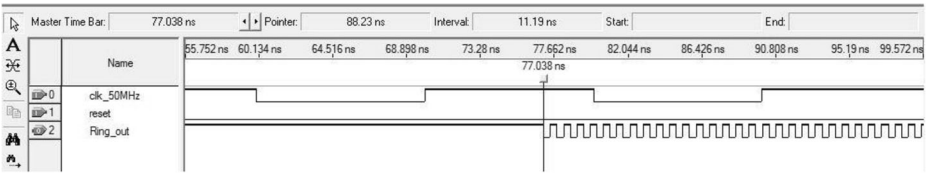

Fig. 3 Timing simulation of three inverter RO

$$f = \frac{1}{ntpd} MHz \tag{3}$$

f – Frequency of oscillations in MHz.
n – Number of inverters in a ring.
t_{pd} – Propagation delay of an inverter.

The frequency generated is indirectly proportional to the number of inverters. Interestingly, the proposed structure yielded an entropy range between 0.92 and 0.97 which is an improved factor as shown in Fig. 7. To create more jitters, a limited number of inverters has been used to construct the TRNG in this design. A continuous aberration in the edges of oscillation causes jitter which is a random and unpredictable phenomenon to be utilised in this true random number generation. The poor statistical properties that can occur due to the biasing condition is a challenge to be overcome.

3.2 Harvesting mechanism

Extraction of randomness from the entropy source requires a proper harvesting mechanism. Harvesting is defined as the process of capturing jitters or any other random mean in which the confused state of perturbations of zeros and ones are sampled. Usually, D Flip-Flop can be used for sampling the bits at a specified frequency. Instead of using de – synchronization technique such as XOR tree, chaotic selection of high-speed jittery clock ensures the number of bit transitions effectively. When XOR tree is used for randomness harvesting, all the oscillations of ROs were XORed together and sampled using DFF. In such case, the direct propagation of XORed jittery clocks causes metastability in DFF where it outputs neither zero nor one [59]. The proposed harvesting technique avoids metastability in DFF and also

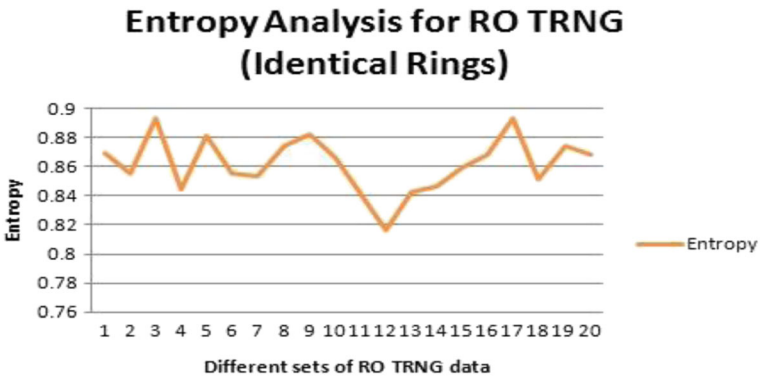

Fig. 4 Entropy analysis of three inverter RO

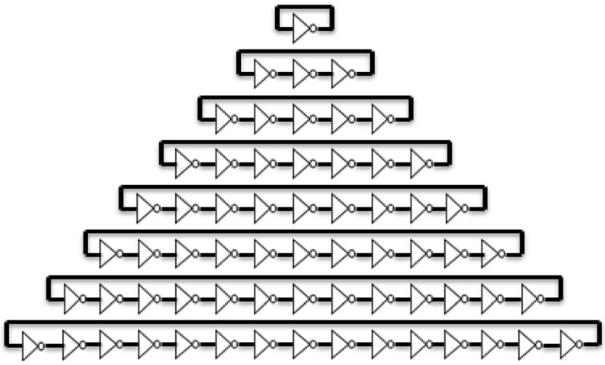

Fig. 5 RO entropy source

diminishes non – synchronized switching activity of true random bits. In this work, harvesting mechanism consists of two individual parts namely Arbitrary Signal Generator and Random Sequence Selector which are combined internally to perform randomness extraction.

3.2.1 Arbitrary signal generator (ASG)

This ASG generates three random signals, and these signals are used for selecting the jittery clocks from the entropy source. The chaotic selection of RO oscillations through ASG yields better outcome during the generation of random bits. The ASG is developed using the rules of Cellular Automata (CA). In general, CA is a collection of cells which can hold one bit of data at a time. The state transition using the combination of adjacent cells is attained by adopting specific connection among the neighbourhood Flip-flops called rules. Each rule can be understood by a unique mathematical expression consisting of previous, present and next cells and also integrated through some logical operations such as AND, OR, NOT, XOR and XNOR. Since the arbitrary signal generator produces random signals, a combination of rules 90 and 150 has been adopted due to their maximal length sequence generation capability and randomness. Rules 90 and 150 are defined through the following Eqs. (4,5) [53],

$$\Psi_{t'} = \Psi_{t-1} \oplus \Psi_{t+1} \tag{4}$$

$$\Psi_{t'} = \Psi_{t-1} \oplus \Psi_{t} \oplus \Psi_{t+1} \tag{5}$$

$\Psi_{t'}$ Next value of a present cell

Ψ_{t-1} present value of the previous cell

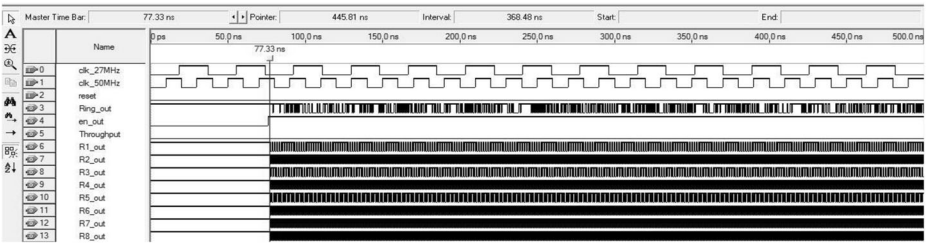

Fig. 6 Timing simulation

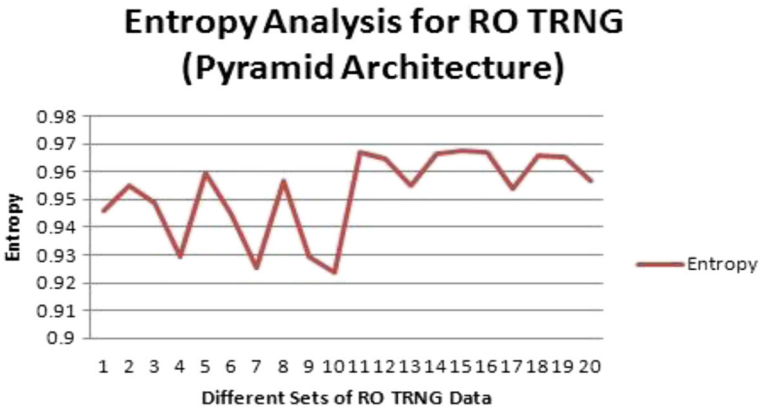

Fig. 7 Entropy analysis of RO Structure

Ψ_{t+1} present value of next cell
Ψ_t present value of present cell

The ASG creates the following selection lines Sel_1, Sel_2 and Sel_3 by XORing the neighbour cells as expressed in the following Eqs. (6–8). Here, Ψ_1, Ψ_2,..... Ψ_{128} denote the Flip-flop outputs in each cell.

$$Sel_1 = \Psi_1 \oplus \Psi_2 \oplus \Psi_3 \oplus \Psi_4 \oplus .....\Psi_{32} \tag{6}$$

$$Sel_2 = \Psi_{33} \oplus \Psi_{34} \oplus \Psi_{35} \oplus \Psi_{36} \oplus .....\Psi_{64} \tag{7}$$

$$Sel_3 = \Psi_{65} \oplus \Psi_{66} \oplus \Psi_{67} \oplus \Psi_{68} \oplus .....\Psi_{96} \tag{8}$$

The Fig. 8 shows the CA structure with the combination of rules 90 and 150 constructed in an alternate fashion with 128 Flip-flops. The simulation of ASG is shown in Fig. 9.

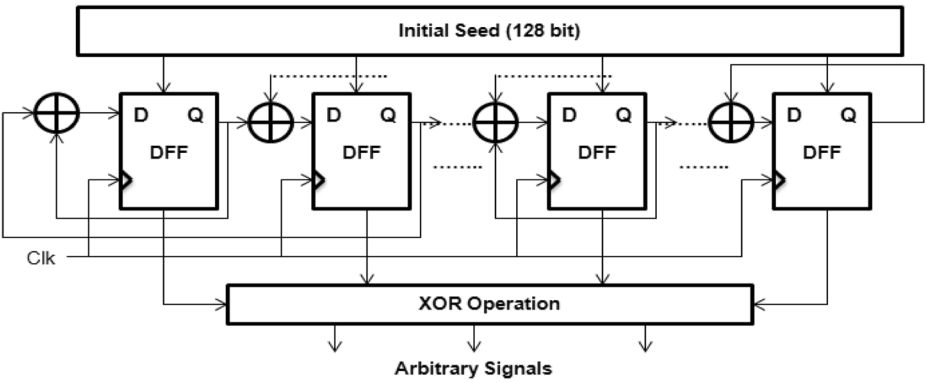

Fig. 8 Block diagram of an arbitrary signal generator

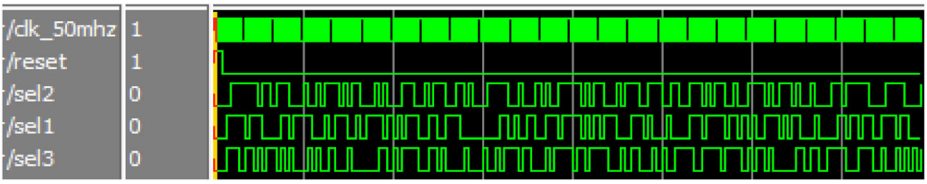

Fig. 9 Simulation of ASG

3.2.2 Architecture of random sequence selector (RSS)

This selector randomly chooses the arbitrary signals with an 8:1 multiplexer. RSS utilises eight jittery clocks as input from RO entropy source and three arbitrary signals Sel_1, Sel_2 and Sel_3 as select lines to produce a sequence of true random jittery bits. The sampling frequency is chosen to be less than or close to the generation frequency of TRNG. Sunar et al. have chosen a sampling frequency of 40 MHz but generated the random bits at a lesser throughput of 2.5 Mbps [51]. In this work, an onboard clock of 27 MHz has opted as sampling frequency H_{clk}. Figure 10 portrays the RTL schematic of RSS and Fig. 11 demonstrates the results of the timing simulation.

3.3 Post-processing & storage

NIST SP 800–22 Suite evaluations decide the strength of any RNG [5]. Even though the modified architecture has reported good entropy, the random bit collections didn't pass some of the NIST tests (Section 4). However, among all other NIST tests, the random bits passed the Rank, FFT, Non-overlapping and linear-complexity tests by a lower margin. There are three major reasons which affect the quality of raw random bits

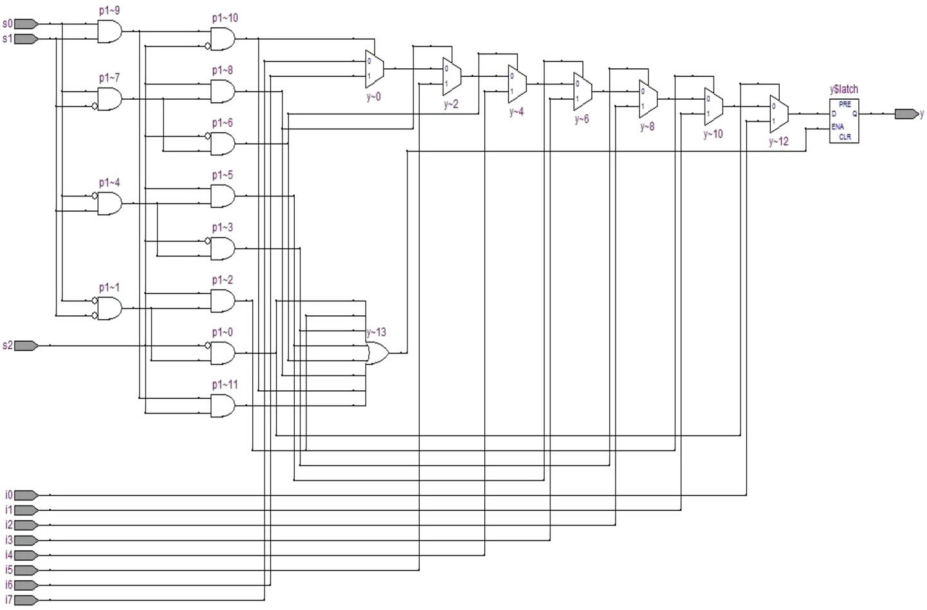

Fig. 10 RTL schematic of Architecture in RSS

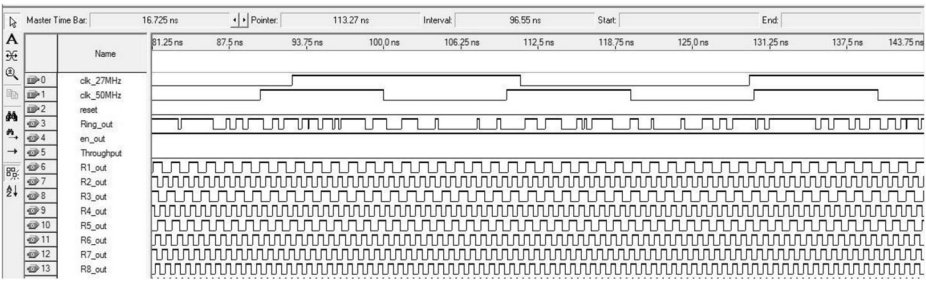

Fig. 11 Timing simulation of RSS

namely poor entropy source, inadequate randomness extraction and correlation in extracted samples. Post-processing is a reliable solution to mitigate these problems. As per the literature, some of the major post-processing techniques utilized are XOR corrector, VNC, resilient functions and hashing. Post-processing techniques support the TRNG to attain sufficient entropy and acceptable statistical characteristics listed by the NIST suite. In this proposed work, two post-processing techniques namely VNC and LFSR have been applied to the random bits and the obtained results were promising. In comparison with VNC, LFSR produced the random bits to be NIST compatible which has been discussed in Section 4.

3.3.1 Von Neumann corrector (VNC)

VNC is a simple and easy post-processing method to reduce the bias in random bits [50]. The main idea of this post-processing is to achieve equal distribution of zeros and ones. It takes two bits at a time from the stream of bit sequence and outputs a bit according to the conditions given in Table 1.

In VNC, two bits with equal logic levels are discarded. Hence, the throughput of TRNG has been reduced by ½ times. In addition, some of the statistical results of NIST suite will not produce an acceptable range of results. The inference is that VNC only improves the entropy per bit but not the other statistical properties of TRNG.

3.3.2 Linear feedback shift register

LFSR has been used as a post-processor in a few works [16, 46]. The random behaviour of LFSR depends on the chosen primitive polynomial for constructing the circuit. LFSR generates random sequences which are further diffused with the raw random sequences of TRNG to produce statistically strong true random numbers. The 128-bit LFSR polynomial used in this work is $X^{128} + X^{126} + X^{101} + X^{99} + 1$ with an initial seed of F3EA9FF87807FFMFFF3EA9FF87807FFFF. Simulation of random numbers generated by LFSR is presented in Fig. 12.

Table 1 VNC conditions

Input 1	Input 2	Output
0	0	Discard bits
0	1	1
1	0	0
1	1	Discard bits

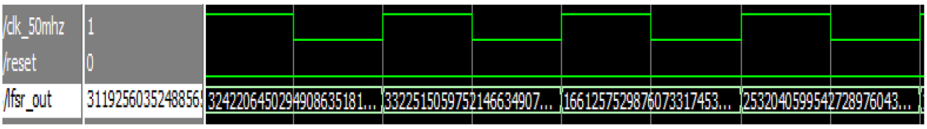

Fig. 12 Random Number generation through LFSR

Equal distribution of 1's and 0's in a sequence of length 'n' is a basic and preliminary test to validate any random number generator. For a good random number generator, the probability of distribution of zeros or ones must be equal to 0.5. The random number generator which doesn't satisfy this basic parameter is subjected to bias.

Let A and B be the two entropy sources with adequate randomness. The expected values named as $E(A) = E(B) = ę$ respectively and let ϕ is said to be its correlation. When the outputs of the entropy source were XORed, the expected value of diffused output can be expressed as,

$$E(A \boxdot B) = \frac{1}{2} - 2\left(ę - \frac{1}{2}\right)^2 - 2\,\phi\, ę(1 - ę) \tag{9}$$

If ę is close to 0.5, this equation can be rewritten as,

$$E(A \boxdot B) \approx \frac{1}{2}(1 - \phi) \tag{10}$$

It is inferred from this expression that when two different random bit sequences were diffused, entropy increases. $E(A \boxdot B) = \frac{1}{2}$ if and only if A and B are linearly independent such that $\phi = 0$. If multiple numbers of independent random bits were diffused say n, it's clear that the expected value ę can be derived as [15],

$$\frac{1}{2} + (-2)^{n-1}\left(ę - \frac{1}{2}\right)^n = \frac{1}{2}\left(1 + (-2\alpha)^n\right) \tag{11}$$

Wherein $\alpha = ę - ½$, ę belongs to (0, 1) which implies $|2\alpha| < 1$. Hence, diffusion of more number of independent random bits causes more bias in the output bits generated. To prevent this bias, DFF based sampling followed by the generation of jittery clocks has been introduced.

Fig. 13 Snapshot of BRAM values

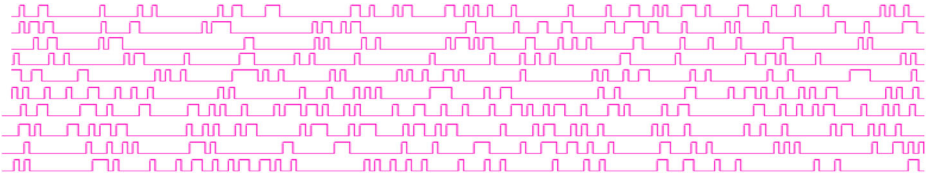

Fig. 14 Restart experiment – RO TRNG

The experiment has been conducted for the RO based TRNG design with two post-processing options namely VNC and LFSR post-processing.

3.4 Memory storage unit

The generated bits can either be stored in on-chip or off-chip memories. The proposed architecture employed on-chip M4KRAM to store the generated random bits. Various 128-bit keys were produced and stored in M4KRAM memory. A total of 1,31,072 true random bits were generated continuously and stored in on-chip Block Random Access Memory (BRAM) of size 128 (No.of bits/key) × 1024 (No.of keys). The memory can be accessed through the inbuilt MegaWizard Plug-In Manager tool of Quartus EDA tool. The sample BRAM values from the proposed TRNG are depicted in Fig. 13.

4 Measurements and statistical test results

The versatility of this TRNG can be proved by checking the implementation of different FPGA families. The proposed design is a generic and device-independent TRNG which was implemented on three different FPGA families namely Altera, Xilinx and Microsemi. Altera Cyclone II EP2C20F484C7, Xilinx 7a100tcsg324–3 and Microsemi M2S150T FPGAs were used for implementation. However, the true random bits obtained from Cyclone II FPGA were considered for analyses. Restart experiment has been conducted to confirm the true randomness of the generator. In addition, NIST tests and entropy analyses have also been performed. Hardware analyses include logic elements consumption, power consumption and throughput.

4.1 Restart experiment

In order to distinguish the true randomness from pseudo randomness, restart experiment is the suitable analysis. Restart experiment for the proposed TRNG has been conducted following the procedures mentioned by Martin H et al. [35]. The importance of this analysis is that it captures the initial bit sequence after reconfiguring the FPGA device. Figures 14, 15,16 demonstrate the five iterations of restart experiment waveforms for RO TRNG without post-

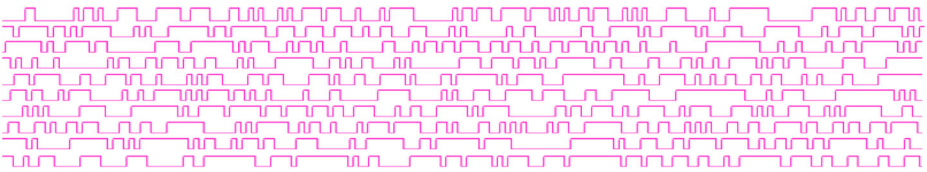

Fig. 15 Restart experiment – RO TRNG with VNC post-processing

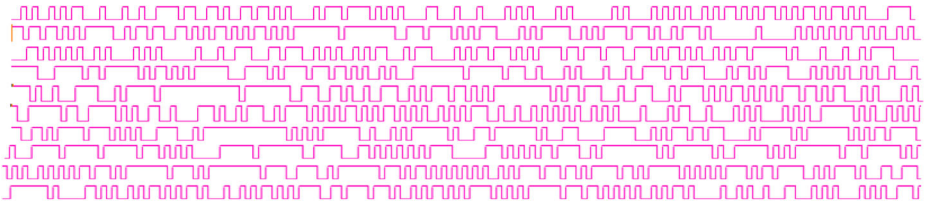

Fig. 16 Restart experiment – RO TRNG with LFSR post-processing

processing, RO TRNG with VNC post-processing and RO TRNG with LFSR post-processing respectively which indicate the non-identical logic level variations as bit generation progresses over time.

4.2 NIST SP 800–22 tests

National Institute of Standards and Technology (NIST) has developed a recommended test suite to evaluate the statistical properties of any random number generator [5]. NIST SP 800-90B states that the entropy source must be an independent noise source or a combination of independent noise sources in which the randomness is to be extracted for producing good true random numbers. In the proposed TRNG design, independent noise sources emerge from each RO of the architecture which produce jittery clock signals. The jitter clocks from all the 8 rings were accumulated and required true random bits were generated subsequently.

The random bits were analysed with NIST SP 800–22 Test Suite by considering 10-bit streams each with a bit length of 10,000. The following parameter settings were applied to various tests before performing NIST randomness analyses.

Block frequency Test – Block length (M) = 128
Non-overlapping Template – Block length (m) = 9
Overlapping Template – Block length (m) = 9
Approximate Entropy Test – Block length (m) = 8

Table 2 NIST SP 800–22 test results of RO TRNG with LFSR post-processing

Test	5 – iterations of P – values				
	I1	I2	I3	I4	I5
Frequency	0.3504	0.9914	0.1223	0.5341	0.7399
Block frequency	0.5341	0.5341	0.1223	0.7399	0.7399
Cumulative Sums – I	0.3504	0.3504	0.2133	0.1223	0.9114
Cumulative Sums – II	0.5341	0.7399	0.3250	0.5341	0.7399
Runs	0.9114	0.2133	0.9114	0.9114	0.1223
Longest Run	0.7399	0.9114	0.1223	0.7399	0.5341
Rank	0.0668	0.3504	0.5341	0.0088	0.2133
FFT	0.3504	0.3504	0.3504	0.9114	0.3504
Non-overlapping Template	0.7399	0.9914	0.9914	0.9114	0.9914
Overlapping Template	0.7399	0.5341	0.7399	0.5341	0.0351
Approximate Entropy	0.3504	0.7399	0.5341	0.1223	0.1223
Serial – I	0.9114	0.3504	0.7399	0.5341	0.0020
Serial – II	0.3504	0.5341	0.1223	0.7399	0.9114
Linear Complexity	0.3504	0.5341	0.1223	0.5341	0.1223

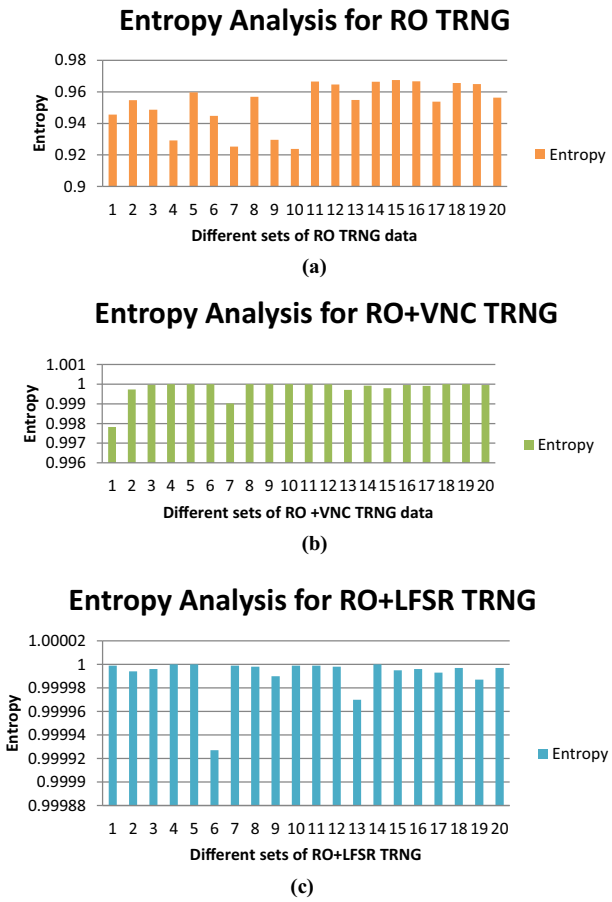

Fig. 17 Entropy analysis – **a** RO based TRNG (**b**) RO with VNC post-processing (**c**) RO with LFSR post-processing

Table 3 Hardware analyses

Target Device	Parameters	Modules and sub - modules of proposed TRNG				
		RSS	ASG	RO TRNG	RO TRNG + VNC	RO TRNG + LFSR
Altera Cyclone II EP2C20F484C7	Logic elements (4 Input LUT + FF)	5	139	627	645	899
	Total power dissipation (mW)	67.10	67.26	67.76	67.75	67.88
	Total memory bits	–	–	1,31,072	1,31,072	1,31,072
Xilinx 7a100tcsg324–3	Logic elements (LUT + FF)	1 + 0	84 + 131	35 + 24	43 + 31	35 + 24
	Total power dissipation (mW)	658	539.9	650	240	650
	Total memory bits	–	–	1,31,072	1,31,072	1,31,072
Microsemi M2S150T	Logic elements (4 Input LUT + FF)	5 + 0	139 + 139	36 + 24	47 + 30	36 + 24
	Total power dissipation (mW)	33.976	34.963	36.112	36.101	36.112
	Total memory bits	–	–	1,31,072	1,31,072	1,31,072

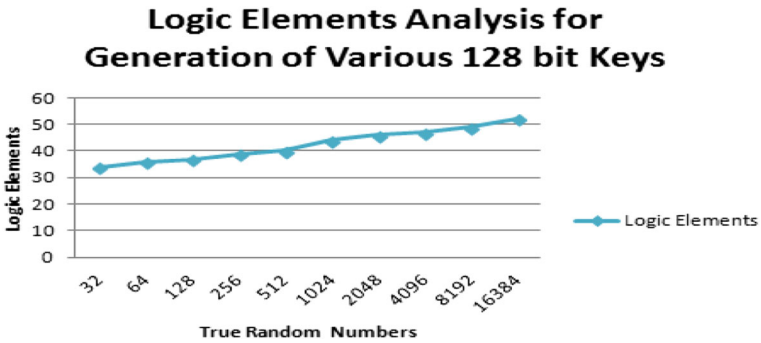

Fig. 18 Logic elements analysis for generation of various 128-bit Keys

Serial Test – Block length (m) = 10
Linear Complexity Test – Block length (M) = 500

NIST SP 800–22 tests have been performed for 20 different sets of true random bits generated through RO TRNG with LFSR post-processing to validate the statistical characteristics, and five sets of results have been given in Table 2 which has yielded good results.

4.3 Entropy analysis

Entropy is one of the essential analyses to ensure the probability of the existence of equivalent occurrences of zeros and ones in a random bit stream. Ideally, the entropy must be equal to 1 with a probability of 0.5 each for a number of zeros and ones. However, a value closer to 1 is required for justifying a good TRNG. Entropy analyses have been carried out for three designs of TRNG, and they are shown in Fig. 17a-c. LFSR post-processing has improved the entropy close to 1 in comparison with VNC and without post-processing cases.

4.4 Hardware analysis

The logic resources, memory utility and power dissipation details are presented in Table 3. It may not be a fair comparison as different FPGA families possess variations in the logic unit and on-chip module architecture. However, device independency is one of the aims of this design which can be verified by the hardware netlist generation capability of the three families chosen for this work. Among the devices, Microsemi dissipates relatively lesser power for hosting the TRNG on M2S150T FPGA. Figure 18 analyses the logic elements consumption for generating 128-bit keys. This analysis was performed on the TRNG without the inclusion of on-chip memory and hence the hardware utility varies between 30 and 50 LEs in Cyclone FPGA for the continuous generation of 2^5 to 2^{14} number of keys depicted in Fig. 18.

Table 4 Throughput analysis

Design	Number of generated bits	Time period (ms)	Throughput (Mbps)
RO TRNG	1,31,072	4.92	26.640650
RO TRNG + VNC	1,31,072	8.36	15.678468
RO TRNG + LFSR	1,31,072	4.92	26.640650

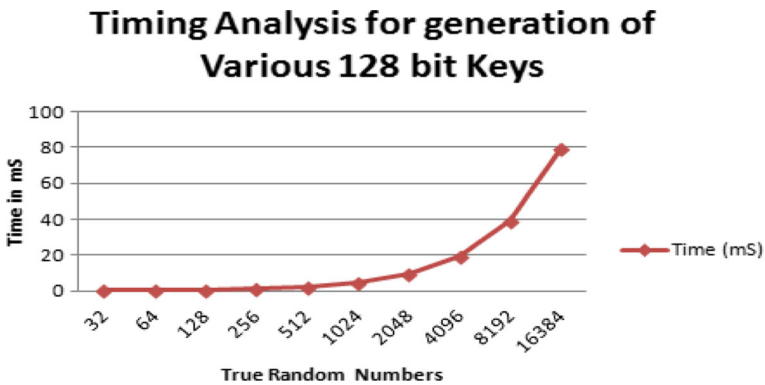

Fig. 19 Timing analysis for generation of various 128-bit Keys

4.5 Throughput analysis

Throughput is the number of bits generated per second by any TRNG. It can be estimated using the following expression.

$$Tp = \frac{Total \quad number \quad of \quad bits \quad generated}{Time \quad taken \quad to \quad generate \quad the \quad bits} \tag{12}$$

Maximum throughput of 26.640650 Mbps with the sampling frequency of 27 MHz has been achieved through the proposed RO based TRNG and Table 4 lists the throughput analysis for TRNGs with post-processing. The proposed TRNG possesses a direct relationship between the sampling frequency and throughput for raw bit generation without post-processing and with LFSR post-processing. Figure 19 shows an exponential increase in the time required to generate 2^5 to 2^{14} numbers of 128-bit keys.

Table 5 Performance comparison - Throughput

Design Category	Reference	Target device	Throughput in Mbps
PLL / DLL	Fischer, Drutarovsky [17]	Altera EP20K200E	0.07
	Fischer et al. [18]	Altera EPIS25	1
	Kwok, Lam [30]	Xilinx XC2VP20	6
	Deak et al. [10]	Xilinx Kintex – 7 XC7K410T	6.25
Free - running oscillator	Kohlbrenner, Gaj [29]	Xilinx XCV1000	0.5
	Schellekens et al. [48]	Xilinx XC2VP30	2.5
	Sunar et al. [51]	Xilinx XC2VP30	2.5
	Hada and Abe [28]	Xilinx XC5VLX50	50
Transition effect ring oscillator	Varchola, Drutarovsky [57]	Xilinx XC3S500E	0.25
Dual metastability	Wieczorek [58]	Digilent's Nexys 2 board	1–5
Coherent sampling	Valchanov et al. [55]	Actel Fusion M7AFS600 and Xilinx XC3S2005ft256	2
Logic gates	Dichtl and Golić [12]	Xilinx XC3S200 – 4FT256C	12.5
Open loop	Danger et al. [9]	Altera Stratix EP1S25	20
RO TRNG + LFSR	Proposed work	Altera Cyclone II EP2C20F484C6	26.640650

Table 6 Performance comparison – Number of inverters

Design category	Reference	Number of ROs		
		Rings	Inverters per ring	Total
Ring Oscillator	Sunar et al. [51]	114	3	342
	Schellekens et al. [48]	110	13	1430
		110	3	330
		210	3	630
	Wold and Tan [59]	50	3	150
	Marghescuet al. [33]	114	3	342
	Ning et al. [38]	1	13	84
		2	17	
		3	23	
		4	31	
	Proposed RO TRNG	8	1 to 15	64

5 Performance analysis

The proposed TRNG is based on RO which is a predominant element used in most of the TRNG designs reported in the literature. The TRNG structure of this work has been modified with 8 rings which have one inverter to 15 inverters at each ring in a variable manner. The experiment of true random number generation was conducted in a laboratory where the temperature was 31.7^0 C measured using HT – 306 thermometer in order to analyse the effect of temperature on TRNG implementation. No constraints were given to the Quartus EDA tool to carry out the placement and routing for implementation of the inverters.

Throughput of the proposed TRNG for a sampling frequency of 27 MHz was computed as 26.640650 Mbps which is higher than the earlier works based on PLL/DLL, metastability, logic gates and open-loop structures (Table 5). The proposed TRNG utilises only 64 inverters and Table 6 depicts the comparison with other RO based TRNGs which proves the compactness of the proposed TRNG design in terms of a number of inverters.

Hence, the proposed design has some advantages over earlier RO based TRNGs which are listed below:

- No PLLs were used
- Independent TRNG irrespective of any FPGA vendor and family
- No complex design constraints were incorporated
- Chaotic selection of random bits to avoid synchronization issues
- Simple post-processing methods
- Optimised entropy source with 64 inverters
- Tamper-resistant TRNG architecture without external influence in TRNG

6 Application to image encryption

Image encryption is the most adopted approach to safeguard the images in an open environment. Confusion and diffusion are the two processes performed in most of the image encryption approaches [44]. TRNGs have been used in the past for image

141	89	227	79	177	175	106	169
142	238	62	78	155	231	184	90
217	111	186	75	29	235	148	179
235	231	238	86	139	45	237	215
159	57	93	125	153	124	239	124
121	23	203	89	46	220	47	163
219	156	137	56	210	156	219	147
223	246	207	167	228	222	45	177

TRNG data at t = T1

14	18	21	36	47	59	7	42
35	27	49	1	5	50	55	4
44	26	11	2	62	19	46	43
56	25	6	63	32	22	48	52
38	28	45	51	57	24	8	10
54	9	61	37	41	39	17	20
31	30	29	34	33	60	40	53
13	58	23	15	64	3	12	16

Indices of sorted TRNG data

144	136	97	85	84	122	133	120
141	171	111	89	171	108	81	157
107	182	110	87	112	184	90	132
71	127	196	120	149	87	70	115
91	161	73	114	151	175	92	118
98	160	118	118	129	77	143	122
151	139	153	167	138	125	109	107
126	125	138	144	128	143	97	123

Confused lena image using TRNG data at t = T1

37	150	216	111	233	162	249	233
236	190	214	78	75	255	217	237
223	158	155	77	103	77	93	189
95	60	145	233	205	73	47	241
167	203	244	237	233	161	38	189
167	201	188	63	185	183	138	185
229	199	139	167	23	191	15	177
77	87	122	167	139	115	203	147

TRNG data at t = T2

55	34	35	64	31	61	18	57
39	8	25	9	32	10	17	2
1	27	48	19	63	47	50	29
53	43	24	11	46	15	3	58
52	26	54	45	38	14	7	60
12	16	23	41	62	13	28	21
30	51	40	5	22	56	33	49
44	4	20	6	59	36	37	42

Indices of sorted TRNG data

81	167	141	128	151	118	136	151
77	92	127	160	149	118	143	87
89	171	70	184	120	84	108	153
107	132	175	110	90	144	143	125
115	182	98	73	91	144	133	125
97	123	138	129	112	126	161	97
139	114	109	171	87	71	138	111
107	157	122	196	122	85	118	120

Confused lena image using TRNG data at t = T2

175	135	90	197	233	117	206	143
255	247	190	57	205	255	196	115
159	220	158	209	156	242	171	153
42	231	155	110	101	125	168	156
199	189	26	122	244	151	199	133
143	91	151	122	231	147	199	63
63	74	181	242	135	219	179	63
143	122	167	111	57	150	123	90

TRNG data at t = T3

21	17	16	6	20	1	53	38
4	64	29	8	35	55	54	33
26	14	30	57	60	23	34	31
40	36	56	46	19	13	49	43
7	28	44	48	3	18	27	37
62	32	61	22	24	50	47	10
63	58	9	45	52	25	11	2
15	41	39	59	51	5	12	42

Indices of sorted TRNG data

97	143	123	196	122	89	107	91
157	128	153	92	141	81	98	138
182	144	139	151	125	138	167	151
109	85	71	90	184	126	111	132
133	161	107	70	143	136	171	118
112	149	118	87	175	108	84	118
120	125	160	73	115	127	110	87
144	129	77	122	114	171	97	120

Confused lena image using TRNG data at t = T3

Fig. 20 Confusion of an 8 × 8 blocks in three ways using TRNG data sets

encryption approaches as well. However, they were utilised only for diffusion operations. Rajagopalan et al. have used RO circuit on ELVIS board to generate a synthetic image for encryption with the RGB image cipher [43]. Ravichandran *et al.* employed PLL based TRNG implemented on FPGA to generate a synthetic image to be used in the encryption of DICOM medical images [45]. Chaotic TRNG was used in another approach proposed by J.Hu and Han [23]. They used multi scroll chaotic attractors to generate true random keys for performing the scrambling operation. However, a confusion process completely based on TRNG has not been proposed yet. TRNG based confusion enables the process of scrambling in any number of ways since it does not follow a fixed sequence pattern. The following few steps can be carried out employing the proposed TRNG for performing confusion of images.

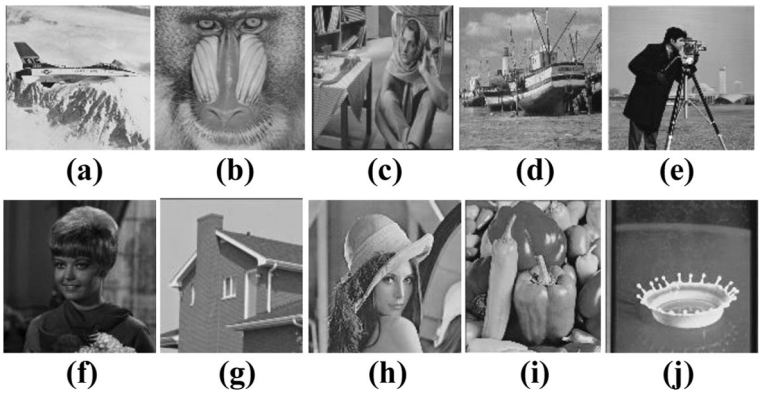

Fig. 21 Test images: (**a**) Airplane; **b** Baboon; **c** Barbara; **d** Boat; **e** Cameraman; **f** Girl; **g** House; **h** Lena; **i** Pepper; **j** Splash

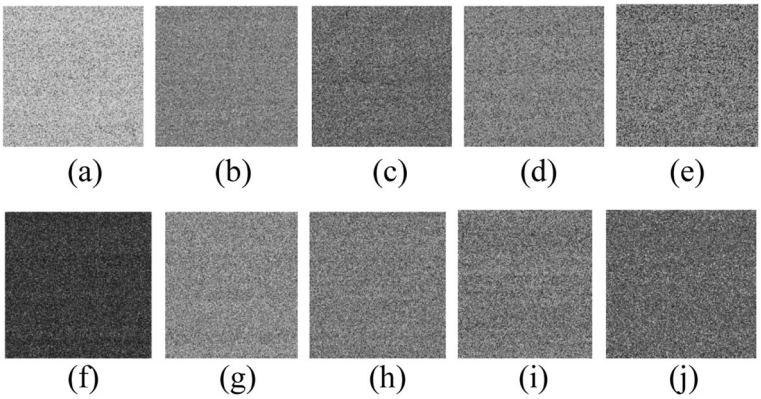

Fig. 22 Scrambled images: (**a**) Airplane; **b** Baboon; **c** Barbara; **d** Boat; **e** Cameraman; **f** Girl; **g** House; **h** Lena; **i** Pepper; **j** Splash

Step 1: Given an Image I(x,y) of size M × N, arrange the pixels as a row/column vector.
Step 2: Generate M × N number of 8-bit key sequence $A = \{A_1, A_2, A_3, \ldots\ldots A_{M \times N}\}$ from the proposed TRNG.
Step 3: Perform sorting function on the TRNG sequence A as follows:

$$[l_A, f_A] = \text{sort (A)};$$

where [•,•] = sort (•) is the sequence indexing function; l_A is the new index after sorting the data A and f_A is the sorted sequence after sorting A.

Step 4: Confuse the binary row / column vector form of I(x,y) in the following form:

$$I(j) \leftrightarrow I(l_A(j)), \qquad where\ 1 \leq j \leq M \times N$$

Figure 20 portrays the confusion process with example pixel blocks chosen from the Lena test grayscale image.

Table 7 Correlation analysis of Original images

Images	H	V	D
Airplane	0.8778	0.8653	0.7847
Baboon	0.8519	0.8698	0.7803
Barbara	0.8254	0.9323	0.7617
Boat	0.8940	0.8930	0.8122
Cameraman	0.9157	0.9540	0.8840
Girl	0.9559	0.9404	0.9174
House	0.9518	0.9325	0.8995
Lena	0.8860	0.9463	0.8456
Peppers	0.9161	0.9375	0.8624
Splash	0.9395	0.9632	0.9152

H Horizontal, *V* Vertical and *D* Diagonal correlation coefficients

Table 8 Correlation analysis of scrambled images

Image	RO TRNG with raw bits			RO TRNG with VNC bits			RO TRNG with LFSR bits		
	H	V	D	H	V	D	H	V	D
Airplane	0.0133	0.0020	0.0133	0.0185	0.0011	−0.0076	0.0100	0.0092	0.0018
Baboon	−0.0017	0.0020	0.0133	0.0054	0.0035	−0.0057	−0.0019	0.0044	0.0065
Barbara	−0.0072	0.0028	0.0119	0.0098	−0.0020	−0.0005	−0.0040	−0.0014	−0.0073
Boat	0.0049	0.0055	−0.0003	0.0116	0.0132	0.0028	−0.0119	−0.0010	0.0095
Cameraman	0.0029	0.0039	0.0070	0.0012	0.0045	0.0067	0.0029	0.0039	0.0070
Girl	0.0202	0.0027	−0.0003	0.0034	0.0078	0.0119	−0.0038	0.0088	0.0020
House	0.0174	0.0015	0.0150	0.0107	0.0163	0.0026	−0.0036	0.0020	0.0122
Lena	0.0134	0.0142	0.0089	0.0192	0.0081	0.0149	0.0017	0.0072	0.0069
Peppers	−0.0128	−0.0141	0.0073	0.0028	0.0017	0.0103	0.0045	0.0230	0.0024
Splash	−0.0005	−0.0000	0.0083	0.0093	−0.0008	0.0046	−0.0025	−0.0082	0.0152

H Horizontal, *V* Vertical and *D* Diagonal correlation coefficients

The prime objective of confusion of image is to attain near-zero correlation among the pixels. Correlation analysis was conducted to validate the strength of the confusion operation influenced by the proposed TRNG. Ten grayscale images of size 128 × 128 were considered from the USC-SIPI image database (Fig. 21a–j). The confusion process was accomplished using LabVIEW 2013 tool after acquiring the TRNG keys from the FPGA, and the obtained images are shown in Fig. 22a–j. Results of correlation analysis for original and scrambled images are presented in Tables 7 and 8.

In general, chaotic maps have been utilized in image encryption for strong confusion. Though they break the relationship between image pixels and ensure low correlation, they are prone to attacks [42]. As a solution, this work proposes a RO TRNG based image confusion through which the near-zero correlation has been observed in most of the test images after confusion process. The effect of TRNG confusion is compared with the other existing chaotic techniques such as logistic map, tent map, combined chaotic map, Lorenz & Lu attractors and the results are presented in Tables 9 and 10. The inference indicates a good sign for utilizing this TRNG based confusion in image encryption schemes.

Figure 23a–c illustrate the correlation plot of 50 confused test images. It was inferred that the horizontal, vertical and diagonal correlation coefficients were close to 0 in the scrambled images.

Table 9 Correlation analysis of scrambled images – Chaotic maps

Image	Logistic Map			Tent Map			Logistic – Sine Map		
	H	V	D	H	V	D	H	V	D
Airplane	−0.0060	0.0030	−0.0026	−0.0043	−0.0051	−0.0127	−0.0028	−0.0037	−0.0063
Baboon	−0.0040	−0.0020	−0.0040	0.0006	0.0105	0.0017	−0.0018	0.0025	−0.0044
Barbara	0.0053	−0.0052	−0.0053	−0.0021	−0.0019	0.0041	−0.0082	−0.0038	0.0015
Boat	0.0050	−0.0063	0.0019	−0.0022	0.0044	−0.0079	0.0083	0.0051	0.0062
Cameraman	−0.0029	0.0001	0.0075	−0.0016	0.0060	−0.0042	−0.0008	−0.0055	−0.0059
Girl	−0.0113	0.0071	−0.0037	−0.0067	−0.0031	0.0027	0.0062	0.0028	−0.0016
House	0.0001	−0.0025	−0.0002	−0.0009	0.0017	0.0071	0.0033	−0.0007	0.0040
Lena	−0.0033	−0.0044	−0.0025	0.0009	−0.0062	−0.0102	0.0031	0.0145	0.0043
Peppers	0.0025	−0.0068	−0.0017	0.0003	−0.0001	−0.0028	0.0001	0.0032	0.0003
Splash	−0.0030	0.0020	−0.0018	0.0072	−0.0043	−0.0046	0.0068	−0.0005	0.0060

Table 10 Correlation analysis of scrambled images – Chaotic attractors

Image	Lorenz Attractor			Lu Attractor		
	H	V	D	H	V	D
Airplane	−0.0014	0.0005	−0.0026	−0.0010	−0.0066	0.0013
Baboon	0.0004	−0.0014	−0.0028	−0.0031	−0.0003	0.0015
Barbara	−0.0009	−0.0019	0.0024	0.0022	−0.0019	−0.0026
Boat	0.0020	−0.0021	−0.0043	0.0042	−0.0026	−0.0021
Cameraman	−0.0012	0.0018	−0.0037	0.0001	−0.0032	−0.0012
Girl	0.0029	0.0029	−0.0016	−0.0003	0.0004	0.0083
House	0.0015	−0.0001	−0.0059	−0.0068	−0.0017	−0.0013
Lena	−0.0042	0.0030	0.0105	−0.0036	0.0047	0.0084
Peppers	−0.0007	−0.0005	0.0013	−0.0046	−0.0056	0.0038
Splash	0.0015	0.0002	−0.0086	0.0025	0.0027	0.0001

7 Conclusion and future work

RO based TRNG with enhanced entropy source and harvesting mechanism has been presented in this work. Chaotic selection of randomness on entropy source with DFF sampling gives an improvement in randomness extraction. Noticeably, the TRNG exhibits better results with LFSR

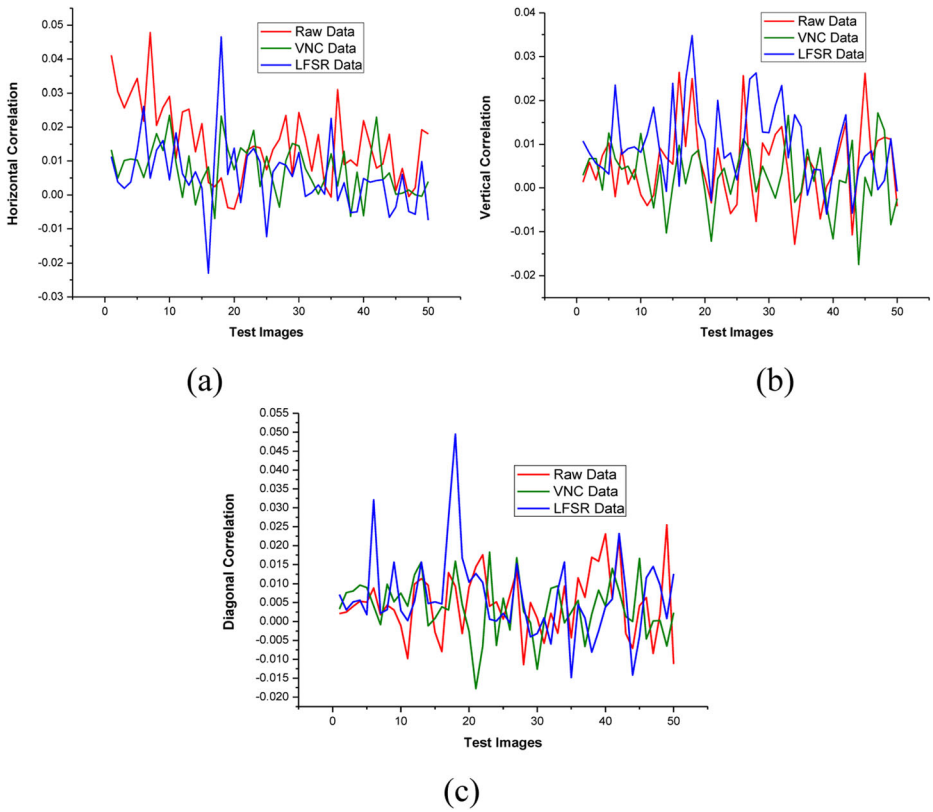

Fig. 23 Correlation plot of 50 test confused images (**a**) Horizontal; (**b**) Vertical (**c**) Diagonal

post-processing in comparison with the earlier works. Statistical characteristics of the proposed TRNG architecture are proven through entropy analysis and NIST tests for 1,31,072 true random bits. Restart experiment confirmed the unpredictable nature of random sequences of TRNG to study the true randomness. The design was implemented on three different FPGA architectures. A throughput of 26.640650 Mbps was achieved which can be improved by increasing the sampling frequency. The proposed design used just 64 inverters in RO rings, relatively less than the previous works. Apart from the utility for key generation, nonce generation, etc., the TRNG's utilisation was extended to the confusion of images which was analysed on ten different standard images yielding an average correlation of −0.000124, 0.000479 and 0.000562 for horizontal, vertical and diagonal planes respectively. In addition, the near-zero correlation was observed through the correlation plot of 50 confused test images. Future work will be on developing a complete image crypto-processor architecture based on RO assisted Physically Unclonable Function (PUF) to provide confidentiality as well as device integrity.

Acknowledgements The authors wish to thank SASTRA Deemed University for providing infrastructure through the Research & Modernization Fund (Ref.No: R&M / 0026 / SEEE – 010 / 2012 – 13) to carry out the research work.

References

1. Arpacı B, Kurt E, Çelik K A new algorithm for the colored image encryption via the modified Chua ' s circuit. Eng Sci Technol an Int J:1–10. https://doi.org/10.1016/j.jestch.2019.09.001
2. Artiles JAP, Chaves DPB, Pimentel C (2019) Image encryption using block cipher and chaotic sequences ☆. Signal Process Image Commun 79:24–31. https://doi.org/10.1016/j.image.2019.08.014
3. Bagini V, Bucci M (1999) A Design of Reliable True Random Number Generator for cryptographic applications. Cryptogr Hardw Embed Syst:204–218. https://doi.org/10.1007/3-540-48059-5_18
4. Barbareschi M, Di Natale G, Torres L (2016) Ring oscillators analysis for security purposes in Spartan-6 FPGAs. Microprocess Microsyst 47:3–10. https://doi.org/10.1016/j.micpro.2016.06.005
5. Bassham LE, Rukhin AL, Soto J, Nechvatal JR, Smid ME, Barker EB, Leigh SD, Levenson M, Vangel M, Banks DL, Heckert NA, Dray JF, Vo S (2010) A statistical test suite for random and pseudorandom number generators for cryptographic applications, Natl. Inst. Stand. Technol., Gaithersburg, MD, USA, Tech. Rep. https://doi.org/10.6028/NIST.SP.800-22r1a
6. Bowman RL (1998) Chapter 20 - Evaluating pseudo-random number generators. In: Pickover CA (ed) Chaos and fractals. Elsevier Science, Amsterdam, pp 133–142. https://doi.org/10.1016/B978-044450002-1/50023-0
7. Bucci M, Germani L, Luzzi R, Tommasino P, Trifiletti A, Varanonuovo M (2003) A high-speed IC random-number source for smartcard microcontrollers. IEEE Trans Circuits Syst I Fundam Theory Appl 50:1373–1380. https://doi.org/10.1109/TCSI.2003.818610
8. Bucci M, Germani L, Luzzi R, Trifiletti A, Varanonuovo M (2003) A high-speed oscillator-based truly random number source for cryptographic applications on a smart card IC. IEEE Trans Comput 52:403–409. https://doi.org/10.1109/TC.2003.1190581
9. Danger JL, Guilley S, Hoogvorst P (2009) High speed true random number generator based on open loop structures in FPGAs. Microelectron J 40:1650–1656. https://doi.org/10.1016/j.mejo.2009.02.004
10. Deak N, Gyorfi T, Marton K, Vacariu L, Cret O (2015) Highly efficient true random number generator in FPGA devices using phase-locked loops, 2015 20th Int. Conf Control Syst Comput Sci:453–458. https://doi.org/10.1109/CSCS.2015.19
11. Denis TS, Johnson S (2007) Chapter 3 - random number generation. In: Denis TS, Johnson S (eds) Cryptogr. Dev. Syngress, Burlington, pp 91–137. https://doi.org/10.1016/B978-159749104-4/50006-6
12. Dichtl M, Golić JD (2007) High-speed true random number generation with logic gates only. Cryptogr Hardw Embed Syst:45–62. https://doi.org/10.1007/978-3-540-74735-2_4
13. Ellis SR (2013) Chapter 2 - a cryptography primer. In: Vacca JR (ed) Comput. Inf. Secur. Handb, 2nd edn. Morgan Kaufmann, Boston, pp 25–46. https://doi.org/10.1016/B978-0-12-394397-2.00002-7

14. Epstein M, Hars L, Krasinski R, Rosner M, Zheng H (2003) Design and implementation of a true random number generator based on digital circuit artifacts. Cryptogr Hardw Embed Syst:152–165. https://doi.org/10.1007/978-3-540-45238-6_13
15. Ergün S, Özoğuz S (2007) Truly random number generators based on a non-autonomous chaotic oscillator. AEU Int J Electron Commun 61:235–242. https://doi.org/10.1016/j.aeue.2006.05.006
16. Fischer V, Bernard F (2011) True Random Number Generators in FPGAs. https://doi.org/10.1007/978-94-007-1338-3
17. Fischer V, Drutarovský M (2003) True Random Number Generator Embedded in Reconfigurable Hardware. In: B.S. Kaliski, Çetin K. Koç, C. Paar (Eds.), Cryptogr. Hardw. Embed. Syst. - CHES 2002 4th Int. Work. Redw. Shores, CA, USA, August 13–15, 2002 Revis. Pap., Springer Berlin Heidelberg, Berlin, Heidelberg, p 415–430. https://doi.org/10.1007/3-540-36400-5_30
18. Fischer V, Drutarovský M, Šimka M, Bochard N (2004) High performance true random number generator in Altera Stratix FPLDs, In: J. Becker, M. Platzner, S. Vernalde (Eds.), F. Program. Log. Appl. 14th Int. Conf. FPL 2004, Leuven, Belgium, August 30–September 1, 2004. Proc., Springer Berlin Heidelberg, Berlin, Heidelberg, p 555–564. https://doi.org/10.1007/978-3-540-30117-2_57
19. Fischer V, Bernard F, Bochard N, Varchola M (2008) Enhancing security of ring oscillator-based TRNG implemented in FPGA, Proc. - 2008 Int. Conf. F. Program. Log. Appl. FPL. 1, 245–250. https://doi.org/10.1109/FPL.2008.4629939
20. Güler Ü, Ergün S (2012) A high speed, fully digital IC random number generator. AEU Int J Electron Commun 66:143–149. https://doi.org/10.1016/j.aeue.2011.06.001
21. Güneysu T, Paar C (2009) Transforming write collisions in block RAMs into security applications, Proc. 2009 Int. Conf. Field-Programmable Technol. FPT'09. 128–134. https://doi.org/10.1109/FPT.2009.5377631
22. Hasheminejad A, Rostami MJ (2019) Optik A novel bit level multiphase algorithm for image encryption based on PWLCM chaotic map. Opt - Int J Light Electron Opt 184:205–213. https://doi.org/10.1016/j.ijleo.2019.03.065
23. Hu J, Han F (2009) A pixel-based scrambling scheme for digital medical images protection. J Netw Comput Appl 32:788–794. https://doi.org/10.1016/j.jnca.2009.02.009
24. Jessa M, Matuszewski L (2011) Enhancing the randomness of a combined true random number generator based on the ring oscillator sampling method, 2011 Int. Conf Reconfigurable Comput FPGAs. 274–279. https://doi.org/10.1109/ReConFig.2011.35
25. Jessa M, Matuszewski L (2012) The use of delay lines in a ring-oscillator-based combined true random number generator, 2012 Int. Conf. Signals Electron. Syst. ICSES 2012 - Conf. Proc. https://doi.org/10.1109/ICSES.2012.6382232
26. Johnson AP, Chakraborty RS, Mukhopadhyay D (2016) An improved DCM-based tunable true random number generator for Xilinx FPGA. IEEE Trans Circuits Syst II Express Briefs 7747:1–1. https://doi.org/10.1109/TCSII.2016.2566262
27. Jun B, Kocher P (1999) The Intel random number generator. Cryptogr Res Inc White Pap 27:1–8
28. Kazumichi H, Koki A (2011) Performance enhancement of the ring oscillator type true random number generator on FPGA. IEICE Tech Rep 2011:1–6 (in Japanese)
29. Kohlbrenner P, Gaj K (2004) An Embedded True Random Number Generator for FPGAs, in: Proc. 2004 ACM/SIGDA 12th Int. Symp. F. Program. Gate Arrays, ACM, New York, NY, USA, p. 71–78. https://doi.org/10.1145/968280.968292.
30. Kwok SHM, Lam EY (2006) FPGA-based High-speed True Random Number Generator for Cryptographic Applications. TENCON 2006–2006 IEEE Reg. 10 Conf., p. 1–4. https://doi.org/10.1109/TENCON.2006.344013
31. Łoza S, Matuszewski Ł, Jessa M (2015) A Random Number Generator Using Ring Oscillators and SHA-256 as Post-Processing. Int J Electron Telecommun 61. https://doi.org/10.1515/eletel-2015-0026
32. Marghescu A, Svasta P (2016) Pushing the Optimization Limits of Ring Oscillator-Based True Random Number Generators, Bica I., Reyhanitabar R. Innov. Secur. Solut. Inf. Technol. Commun. SECITC 2016. Lect. Notes Comput. Sci. Springer, Cham 10006, 209–224. https://doi.org/10.1007/978-3-319-47238-6
33. Marghescu A, Teseleanu G, Maimut DS, Neacsa T, Svasta P (2014) Adapting a ring oscillator-based true random number generator for Zynq system on chip embedded platform, 2014 IEEE 20th Int. Symp. Des. Technol. Electron. Packag. SIITME 2014. 197–202. https://doi.org/10.1109/SIITME.2014.6967027
34. Marghescu A, Vasile DC, Svasta P, Simion E (2016) Personalized ring oscillator-based true random number generator analysis using non-invasive attacks, 2016 IEEE 22nd Int. Symp Des Technol Electron Packag SIITME 2016. 98–101. https://doi.org/10.1109/SIITME.2016.7777253.
35. Martin H, Peris-Lopez P, Tapiador JE, San Millan E (2016) A new TRNG based on coherent sampling with self-timed rings. IEEE Trans Ind Informatics 12:91–100. https://doi.org/10.1109/TII.2015.2502183

36. Naseer Y, Shah D, Shah T (2019) Microprocessors and microsystems a novel approach to improve multimedia security utilizing 3D mixed chaotic map. Microprocess Microsyst 65:1–6. https://doi.org/10.1016/j.micpro.2018.12.003
37. Niccolò Battezzati MV (2011) Luca Sterpone, Reconfigurable Field Programmable Gate Arrays for Mission-Critical Applications
38. Ning L, Ding J, Chuang B, Xuecheng Z (2015) Design and validation of high speed true random number generators based on prime-length ring oscillators. J China Univ Posts Telecommun 22:1–6. https://doi.org/10.1016/S1005-8885(15)60661-6
39. Padua D, ed. (2011) Application-Specific Integrated Circuits. Encycl. Parallel Comput., Springer US, Boston, MA, p 71. https://doi.org/10.1007/978-0-387-09766-4_2205
40. Peeters E, Standaert F-X, Donckers N, Quisquater J-J (2005) Improved higher-order Side-Channel attacks with FPGA experiments. In: Rao JR, Sunar B (eds) Cryptogr. Hardw. Embed. Syst. – CHES 2005. Springer Berlin Heidelberg, Berlin, pp 309–323
41. Pétrie CS, Alvin Connelly J (2000) A noise-based IC random number generator for applications in cryptography. IEEE Trans Circuits Syst I Fundam Theory Appl 47:615–621. https://doi.org/10.1109/81.847868
42. Preishuber M, Hütter T, Katzenbeisser S, Member S, Uhl A (2018) 'Depreciating motivation and empirical security analysis of Chaos-based image and video Encryption'IEEE trans. Inf Forensics Secur 13(9):2137–2150
43. S. Rajagopalan, S. Rethinam, S. Arumugham, H.N. Upadhyay, J.B.B. Rayappan, R. Amirtharajan, Networked hardware assisted key image and chaotic attractors for secure RGB image communication, Multimed. Tools Appl. (2018). doi:https://doi.org/10.1007/s11042-017-5566-0.
44. Ramalingam B, Ravichandran D, Annadurai AA, Rengarajan A, Rayappan JBB (2018) Chaos triggered image encryption - a reconfigurable security solution. Multimed Tools Appl 77:11669–11692. https://doi.org/10.1007/s11042-017-4811-x
45. Ravichandran D, Rajagopalan S, Upadhyay HN, Rayappan JBB, Amirtharajan R (2018) Encrypted biography of biomedical image - a Pentalayer cryptosystem on FPGA. J Signal Process Syst. https://doi.org/10.1007/s11265-018-1337-z
46. Robert GE, Oerlemans VM (2002) United States Patent Application Publication on Digital True Random Number Generator Circuit
47. Robson S, Leung B, Gong G (2014) Truly random number generator based on a ring oscillator utilizing last passage time. IEEE Trans Circuits Syst II Express Briefs 61:937–941. https://doi.org/10.1109/TCSII.2014.2362715
48. Schellekens D, Preneel B, Verbauwhede I (2006) FPGA vendor agnostic true random number generator, Proc. - 2006 Int. Conf. F. Program. Log. Appl. FPL. 139–144. https://doi.org/10.1109/FPL.2006.311206
49. Stipčević M, Koç ÇK (2014) True Random Number Generators. In: Koç ÇK (ed) Open Probl. Math. Comput. Sci. Springer International Publishing, Cham, pp 275–315. https://doi.org/10.1007/978-3-319-10683-0_12
50. Sunar B (2009) True Random Number Generators for Cryptography. https://doi.org/10.1007/978-0-387-71817-0
51. Sunar B, Martin WJ, Stinson DR (2007) A provably secure true random number generator with built-in tolerance to active attacks. IEEE Trans Comput 56:109–119. https://doi.org/10.1109/TC.2007.250627
52. Tkacik TE (2002) A hardware random number generator. Cryptogr Hardw Embed Syst:450–453. https://doi.org/10.1007/3-540-36400-5_32
53. Tomassini M, Perrenoud M (2001) Cryptography with cellular automata. Appl Soft Comput 1:151–160. https://doi.org/10.1016/S1568-4946(01)00015-1
54. Tsoi KH, Leung KH, Leong PHW (2003) Compact FPGA-based true and pseudo random number generators, IEEE Symp. FPGAs Cust. Comput. Mach. Proc. 2003–Janua. 51–61. https://doi.org/10.1109/FPGA.2003.1227241
55. B. Valtchanov, Fischer V, Aubert A (2009) Enhanced TRNG based on the coherent sampling. 3rd Int Conf Signals, Circuits Syst SCS 2009:1–6. https://doi.org/10.1109/ICSCS.2009.5412601
56. Henk CA Van Tilborg, Jajodia S eds. (2011) FPGA Field Programmable Gate Array, in: Encycl. Cryptogr. Secur., Springer US, Boston, MA, p 499. https://doi.org/10.1007/978-1-4419-5906-5_1326
57. Varchola M, Drutarovsky M (2010) New high entropy element for FPGA based true random number generators. In: Mangard S, Standaert F-X (Eds.), Cryptogr. Hardw. Embed. Syst. CHES 2010 12th Int. Work. St. Barbar. USA, August 17–20, 2010. Proc., Springer Berlin Heidelberg, Berlin, Heidelberg, p 351–365. https://doi.org/10.1007/978-3-642-15031-9_24
58. Wieczorek PZ (2013) Dual-metastability FPGA-based true random number generator. Electron Lett 49:744–745. https://doi.org/10.1049/el.2012.4126

59. Wold K, Tan CH (2008) Analysis and enhancement of random number generator in FPGA based on oscillator rings, Proc. - 2008 Int. Conf. Reconfigurable Comput. FPGAs, ReConFig 2008. 2009, 385–390. https://doi.org/10.1109/ReConFig.2008.17.
60. Xingyuan W, Junjian Z, Guanghui C (2019) An image encryption algorithm based on ZigZag transform and LL compound chaotic system. Opt Laser Technol 119:105581. https://doi.org/10.1016/j.optlastec.2019.105581

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Sivaraman R. received his B.Tech (Electronics and Communication Engineering) in 2014 from SRC, SASTRA Deemed University, Kumbakonam and M.Tech in VLSI Design from SASTRA Deemed University, Thanjavur in 2016. He is currently working as a research scholar in the school of electrical and electronics engineering, SASTRA Deemed University, India. His research areas include information security, the design of hardware peripherals and embedded system. Email: sivaraman77uma@sastra.ac.in

Sundararaman Rajagopalan completed his B.Tech in Electronics & Instrumentation Engg, M.Tech in Advanced Communication Systems and PhD in Information Security in the years 2005, 2007 and 2015 respectively. He is currently working as Senior Assistant Professor in the Dept. of ECE, SASTRA Deemed University. His current research interests include Image security, Encryption, Steganography and True Random Number Generator. He has also published more than 35 Research articles in International journals. Email:raman@ece.sastra.edu

Rengarajan Amirtharajan received his B.E. degree from P.S.G. Tech. Bharathiyar University, Coimbatore, India in 1997. He received M.Tech. and Ph.D. degrees from SASTRA University, Thanjavur, India in 2007 and 2012, respectively. He is currently working as an Associate Professor in SEEE, SASTRA University. He has patented a novel embedding scheme USPTO during March 2015. He has also published more than 161 Research articles in National & International journals. Email:amir@ece.sastra.edu